

ESWIN

ESSDK内存用户手册

Rev 1.1

2026-05-11

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

版本号	日期	说明
V1.1	2026-5-11	新增 API ES_UserFd2MemFd 接口及其相关说明
V1.0	2026-4-13	重构文档，添加 python 结构
V0.9.0	2026-1-22	第一节概述部分，添加 nid 和 partition 支持上限的描述，每个 die 至多支持 8 个 partition。附录/proc 调试信息说明，新增 mem_info 查看当前各个 partition 内存使用情况第三节，ES_VB_CreatePool 参数描述更正 poolId 为 output 参数
V0.8.1	2025-12-12	第 2.3 节添加 ES_USER_Malloc_To_MenFd 接口附加说明；第 3 节，添加 ES_USER_Malloc_To_MenFd 接口注意事项
V0.8.0	2025-07-17	第 2 节修改 API 简介 添加 2.3 小节，添加用户态内存分配管理第 3 节，添加 ES_USER_Malloc_To_MenFd 接口说明和定义
V0.7.0	2025-06-24	2.2 小节和第 3 节，新增 ES_VB_registerBlockRefCountNotifyCallback 接口说明和定义第 6 节，proc 调试信息获取目录修改为/proc/eswin/vb
V0.6.1	2025-05-13	2.1 小节，增加 ES_SYS_MemFree 的使用说明 2.2 小节，增加 ES_VB_FreelOVA 的使用说明
V0.6.0	2025-03-20	2.1 小节，增加 ES_SYS_MemFlushAll, ES_SYS_MemFlushAllByNid 接口第 3 节，增加 ES_SYS_MemFlushAll, ES_SYS_MemFlushAllByNid 接口定义，修改 ES_SYS_RetrieveMemNodeByFd 和 ES_SYS_RetrieveMemNodeByVaddr 的 node ID 数据类型。第 4 节，新增 EIC770X_LOGICAL_MEM_NODE_E 类型定义
V0.5.4	2024-11-04	2.1 小节，ES_SYS_MemFlushCache 接口，增加 cache 同步的使用说明
V0.5.3	2024-09-11	修改副标题为"EIC7x 系列 AI 数字 SoC"
V0.5.2	2024-09-09	增加 1.1 小节 配置 MMZ 空间大小
V0.5.1	2024-02-20	Initial edition, an engineering draft.

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	6
1.1. 内存区域	6
1.2. 配置 MMZ 空间大小	6
2. C API 接口说明	11
2.1. 系统内存管理 (ES_SYS)	11
2.1.1. ES_SYS_MemAlloc	11
2.1.2. ES_SYS_Mmap	11
2.1.3. ES_SYS_Munmap	12
2.1.4. ES_SYS_MemFlushCache	13
2.1.5. ES_SYS_MemFree	13
2.1.6. ES_SYS_SplitMemFd	14
2.1.7. ES_SYS_RcvMemFd	14
2.1.8. ES_SYS_GetMemSize	15
2.1.9. ES_SYS_RetrieveMemNodeByFd	15
2.1.10. ES_SYS_RetrieveMemNodeByVaddr	16
2.1.11. ES_SYS_MemFlushCacheAllByNid	16
2.1.12. ES_SYS_MemFlushCacheAll	17
2.2. 视频缓冲管理 (ES_VB)	17
2.2.1. ES_VB_CreatePool	17
2.2.2. ES_VB_DestroyPool	18
2.2.3. ES_VB_SetConfig	18
2.2.4. ES_VB_GetConfig	19
2.2.5. ES_VB_Init	19
2.2.6. ES_VB_Exit	20
2.2.7. ES_VB_SetModPoolConfig	21
2.2.8. ES_VB_GetModPoolConfig	21
2.2.9. ES_VB_ModPoolInit	22
2.2.10. ES_VB_ModPoolExit	22
2.2.11. ES_VB_GetModBlock	23
2.2.12. ES_VB_GetBlock	23
2.2.13. ES_VB_ReleaseBlock	24
2.2.14. ES_VB_AllocIOVA	25
2.2.15. ES_VB_IOVA2Handle	25
2.2.16. ES_VB_FreelOVA	26
2.2.17. ES_VB_Handle2PoolId	26
2.2.18. ES_VB_MmapPool	27
2.2.19. ES_VB_MunmapPool	27
2.2.20. ES_VB_GetBlockVirAddr	28
2.2.21. ES_VB_GetBlockRefCount	28
2.2.22. ES_VB_registerBlockRefCntNotifyCallback	29
2.3. 用户内存转换 (ES_USER)	30
2.3.1. ES_USER_Malloc_To_MemFd	30
2.3.2. ES_UserFd2MemFd	30
2.4. 数据类型	31
2.4.1. SYS_CACHE_MODE_E	31
2.4.2. VB_POOL	31
2.4.3. VB_POOL_CONFIG_S	32
2.4.4. VB_CONFIG_S	32

2.4.5. VB_UID_E	32
2.4.6. EIC770X_LOGICAL_MEM_NODE_E	33
3. Python API 接口说明	34
3.1. API 详细说明	34
3.1.1. memory_alloc()	34
3.1.2. memory_map()	35
3.1.3. memory_unmap()	36
3.1.4. memory_flush_cache()	36
3.1.5. memory_free()	37
3.1.6. memory_split_fd()	37
3.1.7. memory_retrieve_fd()	38
3.1.8. get_memory_size()	38
3.1.9. retrieve_node_by_fd()	39
3.1.10. retrieve_node_by_vaddr()	39
3.1.11. memory_flush_cache_all_by_nid()	40
3.1.12. memory_flush_cache_all()	40
3.1.13. get_internal_fd()	41
3.1.14. get_internal_memory_fd()	41
3.1.15. video_buffer_create_pool()	41
3.1.16. video_buffer_destroy_pool()	42
3.1.17. video_buffer_set_config()	42
3.1.18. video_buffer_get_config()	43
3.1.19. video_buffer_init()	43
3.1.20. video_buffer_exit()	44
3.1.21. video_buffer_set_mod_pool_config()	44
3.1.22. video_buffer_get_mod_pool_config()	44
3.1.23. video_buffer_mod_pool_init()	45
3.1.24. video_buffer_mod_pool_exit()	45
3.1.25. video_buffer_get_mod_block()	46
3.1.26. video_buffer_get_block()	46
3.1.27. video_buffer_release_block()	47
3.1.28. video_buffer_alloc_iova()	48
3.1.29. video_buffer_iova2_handle()	48
3.1.30. iova_capsule_to_int()	49
3.1.31. video_buffer_free_iova()	49
3.1.32. video_buffer_handle2_pool_id()	50
3.1.33. video_buffer_memory_map_pool()	50
3.1.34. video_buffer_memory_unmap_pool()	50
3.1.35. video_buffer_get_block_vir_addr()	51
3.1.36. video_buffer_register_block_ref_cnt_notify_callback()	51
3.1.37. video_buffer_get_block_ref_count()	52
3.1.38. malloc_to_memory_fd()	53
3.2. 数据类型	53
3.2.1. SYS_CACHE_MODE_E	54
3.2.2. VB_POOL	54
3.2.3. VB_POOL_CONFIG_S	54
3.2.4. VB_CONFIG_S	55
3.2.5. VB_UID_E	55
3.2.6. EIC770X_LOGICAL_MEM_NODE_E	55
4. 错误码	57
4.1. SYS 模块错误码	57
4.2. VB 模块错误码	57
5. 附录	59
5.1. 使用说明	59

5.2. Proc 调试信息说明 59

1. 概述

Memory 模块主要向媒体业务提供大块物理内存管理功能，负责内存缓存池。内存块的分配和回收，让物理内存资源在各个媒体处理模块中合理使用。Memory 模块的框架如下图所示：

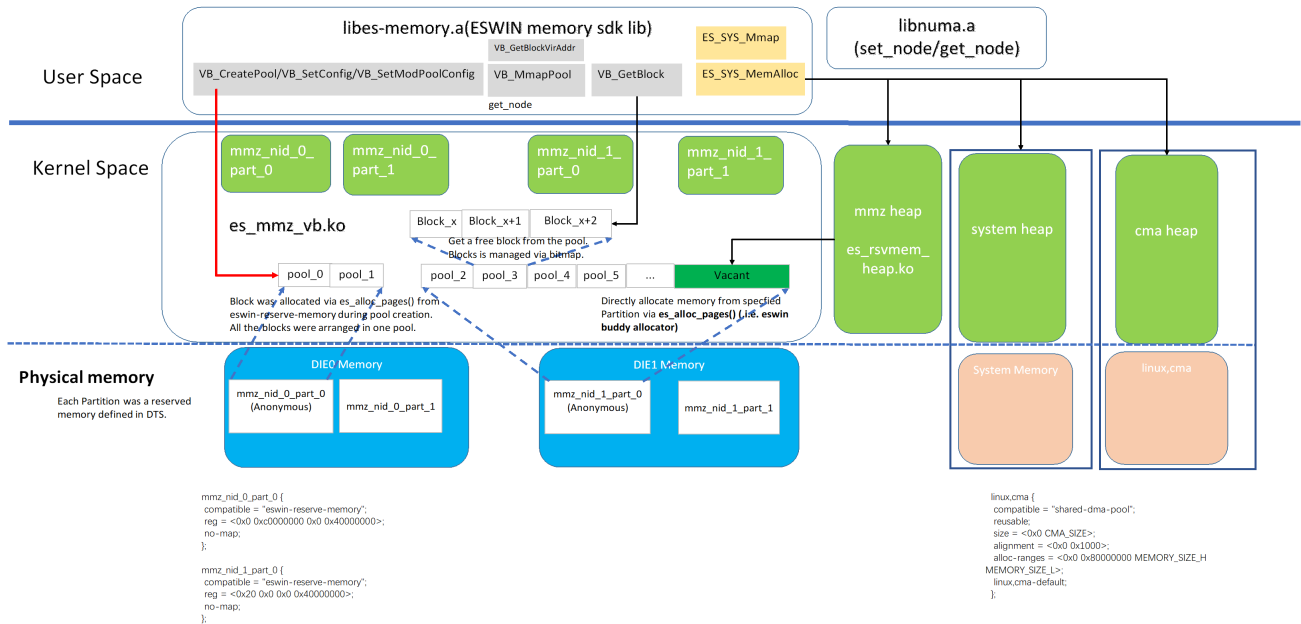


图 1: Memory 模块框架

1.1. 内存区域

Memory 功能模块所管理的内存空间被划分为以下几个区域，用户可以在这些区域分配内存块，每个被分配的内存块有一个引用计数，当引用计数为 0 时，该内存块会被释放回它所在的内存区域。

- **mmz_nid_0_part_n**: die0 上编号为 n 的内存区域，在用户分配 die0 内存块时，如果没指定分配区域，mmz_nid_0_part_0 区域作为默认区域分配内存块，在此区域分配的内存块物理地址不保证连续。
- **mmz_nid_1_part_n**: die1 上编号为 n 的内存区域，在用户分配 die1 内存块时，如果没指定分配区域，mmz_nid_1_part_1 区域作为默认区域分配内存块，在此区域分配的内存块物理地址不保证连续。
- **llc_spram0**: die0 上的 spram 内存区域。
- **llc_spram1**: die1 上的 spram 内存区域。
- **system**: linux 系统自带的 dma heap 内存区域，在此区域分配的内存块物理地址不保证连续。
- **linux,cma**: linux 系统自带的内存区域，在此区域分配的内存块物理地址连续。

注

1. 对于 mmz_nid_x_part_x 内存区域，在此文中统一叫做 MMZ 的 partition。目前至多两个 die，每个 die 至多支持 8 个 partition，也即 mmz_nid_0_part_0, ..., mmz_nid_0_part_7, mmz_nid_1_part_0, ..., mmz_nid_1_part_7。
2. llc_spram 区域最大只有 4M。
3. linux,cma 区域的 ddr 保证物理连续，除此之外的其它区域均不保证物理连续。
4. llc_spram 区域的内存，CPU 访问其 VA 会经过 cache，其他区域的内存则受到内存分配接口的入参控制。
5. Memory 模块设计基于 DMA-BUF，其用户态 lib 库为 libes_memory.so。

1.2. 配置 MMZ 空间大小

MMZ partition，以及 linux,cma 都是 reserved memory 空间，其起始地址以及大小是通过 DTS 配置确定的。用户可根据实际板子的 DDR 容量大小，以及业务的需求，定义它们的起始地址和大小。

提示

请避开 0xc0000000-0xc0200000 区域（该区域存放了 DTB）。

如下 eic77xx.dts 所示：


```

#define MEMORY_SIZE_H 0x3
#define MEMORY_SIZE_L 0x80000000
#define CMA_SIZE 0x10000000

#include "eswin-win2030-die0-soc.dtsi"
#include "eic7700-pinctrl.dtsi"
#include <dt-bindings/interrupt-controller/irq.h>
#include <dt-bindings/gpio/gpio.h>

/* Clock frequency (in Hz) of the PCB crystal for rtcclk */

/ {
    #address-cells = <2>;
    #size-cells = <2>;
    model = "ESWIN EIC7700";
    compatible = "sifive,hifive-unmatched-a00", "sifive,fu740-c000",
        "sifive,fu740", "eswin,eic7700";

    aliases {
        serial0 = &d0_uart0;
        ethernet0 = &d0_gmac0;
        ethernet1 = &d0_gmac1;
    };

    chosen {
        stdout-path = "serial0:115200n8";
    };

    cpus {
        timebase-frequency = <RTCCLK_FREQ>;
    };

    memory@80000000 {
        device_type = "memory";
        reg = <0x0 0x80000000 MEMORY_SIZE_H MEMORY_SIZE_L>;
        numa-node-id = <0>;
    };

    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;

        linux,cma {
            compatible = "shared-dma-pool";
            reusable;
            size = <0x0 CMA_SIZE>;
            alignment = <0x0 0x1000>;
            alloc-ranges = <0x0 0x80000000 MEMORY_SIZE_H MEMORY_SIZE_L>;
            linux,cma-default;
        };
    };
}

```

图 2: eicxx.dts 示例1

```

mmz_nid_0_part_0 {
    compatible = "eswin-reserve-memory";
    reg = <0x1 0x80000000 0x1 0x80000000>;
    no-map;
};

mmz_nid_0_part_1 {
    compatible = "eswin-reserve-memory";
    reg = <0x3 0x0 0x1 0x0>;
    no-map;
};
};

```

图 3: eicxx.dts 示例 2

用户也可以使用 EIC7X uboot, 通过 uboot 命令 `fdt mmz <name> <addr> <size>` 动态增加或者修改 mmz 空间的大小。例如:

```

=> fdt mmz mmz_nid_0_part_1 0x200000000 0x10000000
Added mmz_nid_0_part_1 to reserved-memory node, addr=0x200000000, size=0x10000000
=> fdt print /reserved-memory
reserved-memory {
    #address-cells = <0x00000002>;
    #size-cells = <0x00000002>;
    ranges;
    mmz_nid_0_part_1 {
        reg = <0x00000002 0x00000000 0x00000000 0x10000000>;
        compatible = "eswin-reserve-memory";
        no-map;
    };
    mmode_resv0@80000000 {
        reg = <0x00000000 0x80000000 0x00000000 0x00080000>;
        phandle = <0x0000006f>;
    };
    mmode_resv1@10,0 {
        reg = <0x00000010 0x00000000 0x00000000 0x00400000>;
        phandle = <0x0000006e>;
    };
    region@E0000000 {
        reg = <0x00000000 0xe0000000 0x00000000 0x01000000>;
        no-map;
        phandle = <0x0000006d>;
    };
    lpcpures@DFFF0000 {
        reg = <0x00000000 0xdfff0000 0x00000000 0x00010000>;
        no-map;
        phandle = <0x00000003>;
    };
};

```

图 4: uboot 工具 fdt mmz 修改 mmz 示例

可以将 fdt mmz 配置命令设置到 uboot 环境变量中, 并且在 boot 时运行, 这样每次 uboot 启动都可以动态增加或者修改 mmz 空间大小。例如:

```
=> env print
arch=riscv
baudrate=115200
board=hifive_premier_p550
board_info=SF106SKB2425000128
board_name=hifive_premier_p550
bootcmd=run mmcboot;run bootkernel;
bootdelay=2
bootfile=SF106SKB2425000128/bootloader_uboot.bin
bootkernel=bootm 0x90000000
cpu=eic7700
dnsip=10.10.2.32
emmc_dev=0
ethladdr=94:47:b0:18:43:48
ethact=ethernet@50400000
ethaddr=94:47:b0:18:43:47
fdt_addr=ed916150
fdt_addr_r=0x88000000
fdt_high=0xffffffffffffffff
fdtaddr=ed916150
fdtcontroladdr=ed916150
fdtfile=eswin/eic7700-hifive-premier-p550.dtb
fileaddr=90000000
filesize=3c0108
gatewayip=10.10.192.1
gpt_partition=gpt write mmc ${emmc_dev} $partitions
initrd_high=0xffffffffffffffff
ipaddr=10.10.203.145
kernel_addr_r=0x84000000
kernel_comp_addr_r=0xa0000000
kernel_comp_size=0x4000000
loadaddr=0x80200000
mmcargs=setenv bootargs root=${mmccroot}
mmcboot=echo Booting from mmc ...;run mmcargs;run mmz_resize;run setimageload;run loadimage
mmccroot=/dev/mmcblk0p3 rootfstype=ext4 rootwait
mmz_resize=fdt mmz mmz nid 0 part 1 0x200000000 0x10000000
```

图 5: uboot 参数默认修改 mmz 示例

2. C API 接口说明

Memory 模块的 C 语言接口提供系统内存分配、映射、缓存维护以及视频缓冲池管理能力。本章节按 ES_SYS 与 ES_VB 两部分介绍相关接口与数据类型。

2.1. 系统内存管理 (ES_SYS)

2.1.1. ES_SYS_MemAlloc

【函数体】

```
ES_S32 ES_SYS_MemAlloc(ES_U64 *memFd, SYS_CACHE_MODE_E cacheMode, const ES_CHAR *pstrMmb,
                        const ES_CHAR* strZone, ES_U64 len);
```

【描述】

从指定的内存区域分配内存块。

【参数】

参数名称	描述	输入/输出
memFd	分配内存后返回的 memFd	输出
cacheMode	内存存在内核态的 cache 属性，参见 SYS_CACHE_MODE_E	输入
pstrMmb	Mmb 名称的字符串，用于 proc 时的信息呈现	输入
strZone	分配内存所对应的区域名（可用的区域名参见 内存区域 ），填 NULL 则默认使用“mmz_nid_0_part_0”	输入
len	内存大小	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 分配内存的区域名只能是前文中描述的内存区域。
- 调用内存分配接口分配出来的内存 buffer 用一个无符号 64 位类型的 memFd 表示，memFd 在进程内唯一。跨进程需要调用 [ES_SYS_RcvMemFd](#) 进行转换得到另外进程下对应的 memFd。
- 内存来自 llc_spram0 时，内核态和用户态访问都不会经过 cache，即 ES_SYS_MemAlloc 和 ES_SYS_Mmap 的 cacheMode 无效。
- 内存来自 linux,cma 时，分配的是物理地址连续的 ddr memory；来自其他区域时，分配的是物理地址非连续的 ddr memory。

2.1.2. ES_SYS_Mmap

【函数体】

```
ES_VOID* ES_SYS_Mmap(ES_U64 memFd, ES_U64 size, SYS_CACHE_MODE_E cacheMode);
```

【描述】

映射指定内存块，得到用户态虚拟地址。

【参数】

参数名称	描述	输入/输出
memFd	需要映射的 memFd	输入
size	需要映射的内存大小	输入
cacheMode	内存在用户态的 cache 属性，参见 SYS_CACHE_MODE_E	输入

【返回值】

返回值	描述
NULL	失败，返回空指针
非 NULL	用户态虚拟地址

提示

ES_SYS_Mmap 和 [ES_SYS_Munmap](#) 必须成对调用。调用 ES_SYS_Mmap 会使内存 buffer 引用计数 blockRefCount 加 1，调用 ES_SYS_Munmap 会使 blockRefCount 减 1。

2.1.3. ES_SYS_Munmap

【函数体】

```
ES_S32 ES_SYS_Munmap(ES_VOID *pVirAddr, ES_U64 size);
```

【描述】

反映射指定内存块的用户态虚拟地址。

【参数】

参数名称	描述	输入/输出
pVirAddr	待操作内存所对应的用户态虚拟地址	输入
size	内存大小：通过 ES_SYS_Mmap 函数 map 的大小	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.1.4. ES_SYS_MemFlushCache

【函数体】

```
ES_S32 ES_SYS_MemFlushCache(ES_U64 memFd);
```

【描述】

刷新 cache 里的内容到内存并使 cache 里的内容无效。

【参数】

参数名称	描述	输入/输出
memFd	待操作内存所对应的 memFd	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 当通过 ES_SYS_Mmap 映射得到 CACHED_BUF 属性的用户态虚拟地址，CPU 访问此虚拟地址，需调用 ES_SYS_MemFlushCache 做好数据同步。即：当 CPU 完成对该内存的写操作后，应调用 ES_SYS_MemFlushCache，以保证内容被写入内存；当 Device 完成对内存块的写入操作，CPU 读操作前，应调用 ES_SYS_MemFlushCache，以保证读到的内容与内存里的内容一致。
- Flush cache 的范围是 memFd 指向的整个内存。

2.1.5. ES_SYS_MemFree

【函数体】

```
ES_S32 ES_SYS_MemFree(ES_U64 memFd);
```

【描述】

释放通过 `ES_SYS_MemAlloc` 申请得到的内存块。

【参数】

参数名称	描述	输入/输出
memFd	内存块的 memFd	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

使用此接口释放 FD 之前，需要先释放该 FD 关联的 IOVA，否则后续 IOVA 无法释放会产生内存泄露。

2.1.6. ES_SYS_SplitMemFd

【函数体】

```
ES_S32 ES_SYS_SplitMemFd(ES_U64 memFd, ES_U64 offset, ES_U64 len,
                          ES_U64 *splittedMemFd);
```

【描述】

从一个内存块的 offset 偏移处，按照指定的 len 大小，生成一个新的内存块。

【参数】

参数名称	描述	输入/输出
memFd	缓存块的 memFd	输入
offset	splittedMemFd 在缓存块的偏移地址，需按 4K PAGE_SIZE 对齐	输入
len	splittedMemFd 的大小，按 PAGE_SIZE 对齐	输入
splittedMemFd	返回 split 的 memFd	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 不支持对 splitted memFd 再调用该 API 进一步 Split。
- 调用 ES_SYS_SplitMemFd，会使传入的 memFd 计数加 1，当 splittedMemFd 释放时，memFd 计数会自动减 1。

2.1.7. ES_SYS_RcvMemFd

【函数体】

```
ES_S32 ES_SYS_RcvMemFd(ES_U64 sndMemFd, ES_U64 *rcvdMemFd);
```

【描述】

在不同进程或者同进程不同软件模块之间传递内存块。

【参数】

参数名称	描述	输入/输出
sndMemFd	发送端的 MemFd	输入
rcvdMemFd	接收端收到的 MemFd	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 接收到新的 rcvdMemFd，在使用完后，需要调用 [ES_SYS_MemFree](#) 或者 [ES_VB_ReleaseBlock](#) 接口释放。
- 所有内存分配接口得到的 memFd 都适用于 rcvdMemFd。
- 调用此接口，会使对应内存 buffer 的引用计数加 1。

2.1.8. ES_SYS_GetMemSize

【函数体】

```
ES_S32 ES_SYS_GetMemSize(ES_U64 memFd, ES_U64 *size);
```

【描述】

获取指定内存块的大小。

【参数】

参数名称	描述	输入/输出
memFd	缓存块的 memFd	输入
size	缓冲块大小	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.1.9. ES_SYS_RetrieveMemNodeByFd

【函数体】

```
ES_S32 ES_SYS_RetrieveMemNodeByFd(ES_U64 memFd, EIC770X_LOGICAL_MEM_NODE_E *nid);
```

【描述】

获取指定内存块所在的 node ID。

【参数】

参数名称	描述	输入/输出
memFd	缓存块的 memFd	输入
nid	node ID	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.1.10. ES_SYS_RetrieveMemNodeByVaddr**【函数体】**

```
ES_S32 ES_SYS_RetrieveMemNodeByVaddr(void *pVirAddr, EIC770X_LOGICAL_MEM_NODE_E *nid);
```

【描述】

通过缓冲块虚拟地址获取缓存块所在的 node ID。

【参数】

参数名称	描述	输入/输出
pVirAddr	缓存块的虚拟地址	输入
nid	node ID	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.1.11. ES_SYS_MemFlushCacheAllByNid**【函数体】**

```
ES_S32 ES_SYS_MemFlushCacheAllByNid(EIC770X_LOGICAL_MEM_NODE_E nid);
```

【描述】

将指定 node ID 所有 L3 缓存内容刷入主存并将缓存内容无效。

【参数】

参数名称	描述	输入/输出
nid	node ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

这里的 nid 是逻辑 nid，参见 [EIC770X_LOGICAL_MEM_NODE_E](#)。

2.1.12. ES_SYS_MemFlushCacheAll

【函数体】

```
ES_S32 ES_SYS_MemFlushCacheAll();
```

【描述】

将所有 L3 缓存内容刷入主存并将缓存内容无效。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2. 视频缓冲管理 (ES_VB)

2.2.1. ES_VB_CreatePool

【函数体】

```
ES_S32 ES_VB_CreatePool(VB_POOL_CONFIG_S *pstVbPoolCfg, VB_POOL *poolId);
```

【描述】

从指定的内存区域创建一个私有视频缓存池。

【参数】

参数名称	描述	输入/输出
pstVbPoolCfg	缓存池配置属性参数指针	输入
poolId	缓存池 ID	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

用户可以指定缓存池 pool 的属性，其中 enRemapMode 设置了 pool 在内核态的虚拟地址是 cache 还是 uncached。

2.2.2. ES_VB_DestroyPool

【函数体】

```
ES_S32 ES_VB_DestroyPool(VB_POOL poolId);
```

【描述】

销毁一个私有视频缓存池。

【参数】

参数名称	描述	输入/输出
poolId	缓存池 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 如果调用时，pool 对应的 block 已释放完，pool 将立即被销毁。
- 如果调用时，pool 还有未释放的 block，pool 将处于待销毁状态，将不能从 pool 中申请新的 block，待所有 block 都释放完之后，该 pool 将会被销毁。

2.2.3. ES_VB_SetConfig

【函数体】

```
ES_S32 ES_VB_SetConfig(const VB_CONFIG_S *pVbConfig);
```

【描述】

设置公共视频缓存池属性。

【参数】

参数名称	描述	输入/输出
pVbConfig	公共视频缓冲池属性指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

此接口可以重复调用，以最后一次设置的为准。

2.2.4. ES_VB_GetConfig

【函数体】

```
ES_S32 ES_VB_GetConfig(VB_CONFIG_S *pVbConfig);
```

【描述】

获取公共视频缓存池属性。

【参数】

参数名称	描述	输入/输出
pVbConfig	公共视频缓冲池属性指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.5. ES_VB_Init

【函数体】

```
ES_S32 ES_VB_Init(ES_VOID);
```

【描述】

创建公共视频缓存池。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 必须先调用 ES_VB_SetConfig，否则会返回失败。
- 此接口不能重复调用，重复调用返回失败。
- 为了保证进程间安全共享，模块缓存池创建后，如果不再使用，用户必须调用 ES_VB_Exit 释放，进程哪怕异常结束也不会自动释放，用户如果发现模块缓存池未释放，可以
 1. 直接复用现有的模块缓存池，直接 ES_VB_GetBlock

```
(void) ES_VB_SetConfig(&stVbConfig);  
// using common pool  
ret = ES_VB_GetBlock(VB_INVALID_POOLID, blkSize, strZone, &memFd);  
if (ret != ES_SUCCESS) {  
    // error handle  
}  
// using memFd
```

2. 须确保当前没有进程/设备在使用该模块缓存池的内存之后调用 ES_VB_Exit 释放之后才能重新配置缓存池属性已经重新初始化模块缓存池。即

```
if (ES_VB_SetConfig(&stVbConfig)) {  
    // assert exit pool safe  
    // then destroy the old one  
    ES_VB_Exit();  
}  
  
ES_VB_SetConfig(&stVbConfig);  
ES_VB_Init();
```

2.2.6. ES_VB_Exit

【函数体】

```
ES_S32 ES_VB_Exit(ES_VOID);
```

【描述】

销毁公共视频缓存池以及清理之前配置的公共缓存池属性。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.7. ES_VB_SetModPoolConfig

【函数体】

```
ES_S32 ES_VB_SetModPoolConfig(VB_UID_E enVbUid, const VB_CONFIG_S *pVbConfig);
```

【描述】

设置模块视频缓存池属性。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入
pVbConfig	模块视频缓冲池属性指针	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

此接口同一个 uid 不支持反复调用。

2.2.8. ES_VB_GetModPoolConfig

【函数体】

```
ES_S32 ES_VB_GetModPoolConfig(VB_UID_E enVbUid, VB_CONFIG_S *pVbConfig);
```

【描述】

获取模块视频缓存池属性。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入
pVbConfig	模块视频缓冲池属性指针	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.9. ES_VB_ModPoolInit

【函数体】

```
ES_S32 ES_VB_ModPoolInit(VB_UID_E enVbUid);
```

【描述】

创建模块视频缓存池。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 须先调用 ES_VB_SetModPoolConfig，否则会返回失败。
- 为了保证进程间安全共享，模块缓存池创建后，如果不再使用，用户必须调用 ES_VB_ModPoolExit 释放，进程哪怕异常结束也不会自动释放，用户如果发现模块缓存池未释放，可以
 1. 直接复用现有的模块缓存池，跳过 ES_VB_ModPoolInit 直接 ES_VB_GetModBlock

```
(void) ES_VB_SetModPoolConfig(enVbUid, &stVbConfig);
ret = ES_VB_GetModBlock(enVbUid, blkSize, strZone, &memFd);
if (ret != ES_SUCCESS) {
    //error handle
}
//using memFd
```

2. 须确保当前没有进程/设备在使用该模块缓存池的内存之后调用 ES_VB_ModPoolExit 释放之后才能重新配置缓冲池属性已经重新初始化模块缓存池。即

```
//setModConfig fail
if (ES_VB_SetModPoolConfig(enVbUid, &stVbConfig)) {
    //assert exit pool safe
    //then destroy the old one
    ES_VB_ModPoolExit(enVbUid);
}

ES_VB_SetModPoolConfig(enVbUid, &stVbConfig);
ES_VB_ModPoolInit(enVbUid);
```

2.2.10. ES_VB_ModPoolExit

【函数体】

```
ES_S32 ES_VB_ModPoolExit(VB_UID_E enVbUid);
```

【描述】

销毁模块视频缓存池以及清理此 UID 之前设置的缓存池属性。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.11. ES_VB_GetModBlock

【函数体】

```
ES_S32 ES_VB_GetModBlock(VB_UID_E enVbUid, ES_U64 blkSize, const ES_CHAR *strZone,
                          ES_U64 *memFd);
```

【描述】

从模块缓存池中获取一个缓存块。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入
blkSize	缓存块大小，以 byte 为单位	输入
strZone	Mmz 的 partition 名字	输入
memFd	缓存块的 memFd	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.12. ES_VB_GetBlock

【函数体】


```
ES_S32 ES_VB_GetBlock(VB_POOL poolId, ES_U64 blkSize, const ES_CHAR *strZone,
    ES_U64 *memFd);
```

【描述】

获取一个缓存块。

【参数】

参数名称	描述	输入/输出
poolId	缓存池 ID 号	输入
blkSize	缓存块大小，以 byte 为单位	输入
strZone	Mmz 的 partition 名字	输入
memFd	缓存块的 memFd	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 用户可以在创建一个缓存池之后，调用本接口从该缓存池中获取一个缓存块；将第 1 个参数 Pool 设置为创建的缓存池 ID；第 2 个参数 blkSize 须小于或等于创建该缓存池时指定的缓存块大小。从指定缓存池获取缓存块时，参数 strZone 无效。
- 如果用户需要从任意一个公共缓存池中获取一块指定大小的缓存块，则可以将第 1 个参数 PoolId 设置为无效 ID 号 (VB_INVALID_POOLID)，将第 2 个参数 blkSize 设置为需要的缓存块大小，并指定要从哪个 strZone 上的公共缓存池获取缓存块。如果指定的 strZone 上并没有公共缓存池，那么将获取不到缓存块。如果 strZone 等于 NULL，则从 mmz_nid_0_part_0 上的公共缓存池获取缓存块。

2.2.13. ES_VB_ReleaseBlock

【函数体】

```
ES_S32 ES_VB_ReleaseBlock(ES_U64 memFd);
```

【描述】

释放一个已经获取的缓存块。

【参数】

参数名称	描述	输入/输出
memFd	缓存块的 memFd	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.14. ES_VB_AllocIOVA

【函数体】

```
ES_S32 ES_VB_AllocIOVA(ES_U64 memFd, VB_UID_E enVbUid, ES_VOID **ppIOVA);
```

【描述】

获取一个缓存块的 IOVA 地址。

【参数】

参数名称	描述	输入/输出
memFd	缓存块的 memFd	输入
enVbUid	模块 ID	输入
ppIOVA	获取的对应模块的 IOVA 地址	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- ES_VB_AllocIOVA 和 ES_VB_FreeIOVA 必须成对调用。
- 若是第一次调用 ES_VB_AllocIOVA，从 enVbUid 模块获取 iova，则缓存块的引用计数 blockRefCount 会加 1，最后一个成对调用的 ES_VB_FreeIOVA 会使 blockRefCount 减 1。
- 此接口正确返回的前提是对应模块的驱动已正确加载。

2.2.15. ES_VB_IOVA2Handle

【函数体】

```
ES_S32 ES_VB_IOVA2Handle(VB_UID_E enVbUid, ES_VOID *pIOVA, ES_U64 *memFd);
```

【描述】

获取模块 IOVA 地址对应的缓存块 memFd。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入

参数名称	描述	输入/输出
pIOVA	IOVA 地址	输入
memFd	模块 IOVA 地址对应的缓存块 memFd	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.16. ES_VB_FreeIOVA

【函数体】

```
ES_S32 ES_VB_FreeIOVA(VB_UID_E enVbUid, ES_U64 memFd);
```

【描述】

释放缓存块的 IOVA 地址空间。

【参数】

参数名称	描述	输入/输出
enVbUid	模块 ID	输入
memFd	缓存块 memFd	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

使用此接口释放 IOVA 之前，必须保证依赖的 memFd 有效，否则无法释放 IOVA，会产生泄露。

2.2.17. ES_VB_Handle2PoolId

【函数体】

```
ES_S32 ES_VB_Handle2PoolId(ES_U64 memFd);
```

【描述】

获取一个缓存块所在缓存池的 ID。

【参数】

参数名称	描述	输入/输出
memFd	缓存块 memFd	输入

【返回值】

返回值	描述
大于等于 0	缓存块所在缓存池的 ID
小于 0	失败，参见 错误码

2.2.18. ES_VB_MmapPool

【函数体】

```
ES_S32 ES_VB_MmapPool(VB_POOL poolId);
```

【描述】

为一个视频缓存池映射用户态虚拟地址。

【参数】

参数名称	描述	输入/输出
poolId	缓存池的 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

通过 ES_VB_MmapPool 映射得到 pool 用户态虚拟地址，在用户态访问该虚拟地址都是 uncached。

2.2.19. ES_VB_MunmapPool

【函数体】

```
ES_S32 ES_VB_MunmapPool(VB_POOL poolId);
```

【描述】

为一个视频缓存池解除用户态映射。

【参数】

参数名称	描述	输入/输出
poolId	缓存池的 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.20. ES_VB_GetBlockVirAddr

【函数体】

```
ES_S32 ES_VB_GetBlockVirAddr(VB_POOL poolId, ES_U64 memFd, ES_VOID **ppVirAddr);
```

【描述】

获取一个视频缓存池中的缓存块的用户态虚拟地址。

【参数】

参数名称	描述	输入/输出
poolId	缓存池的 ID	输入
memFd	缓冲块的 memFd	输入
ppVirAddr	缓存块的虚拟地址	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

使用之前必须已调用 ES_VB_MmapPool。

2.2.21. ES_VB_GetBlockRefCount

【函数体】

```
ES_S32 ES_VB_GetBlockRefCount(ES_U64 memFd, ES_U64 *blkFileCnt);
```

【描述】

获取缓冲块所对应 block 在内核态的引用计数。

【参数】

参数名称	描述	输入/输出
memFd	缓存块 memFd	输入
blkFileCnt	Block 的引用计数	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

2.2.22. ES_VB_registerBlockRefCntNotifyCallback

【函数体】

```
ES_S32 ES_VB_registerBlockRefCntNotifyCallback(ES_U64 memFd, ES_U64 notifyRefcnt,
        VB_BlockRefCntCallback callback,
        void *callback_arg);
```

【描述】

为指定 memFd 注册一个 notifier，当引用计数减小到 notifyRefCnt 时，触发此 notifier。

【参数】

参数名称	描述	输入/输出
memFd	缓存块 memFd	输入
notifyRefcnt	目标引用计数值	输入
callback	函数指针，需要执行的 notifier	输入
callback_arg	Callback 的入参	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

此功能主要是提供：当 dma_buffer 需要共享给其它任务时，用户可以感知到这些任务已经结束，并唤醒此 notifier 进行下一步工作。

- ES_SYS_SplitMemFd 接口获取的 memFd 不支持注册 callback。
- 一个 Fd 不支持重复注册 callback。
- 需要跨进程共享的 memFd 不支持注册 callback。

2.3. 用户内存转换 (ES_USER)

2.3.1. ES_USER_Malloc_To_MemFd

【函数体】

```
ES_S32 ES_USER_Malloc_To_MemFd(ES_VOID *orig_ptr, ES_U64 len, ES_U64 *dma_fd,
                                SYS_CACHE_MODE_E cacheMode);
```

【描述】

将用户态 malloc 的内存转换为内存块。

【参数】

参数名称	描述	输入/输出
orig_ptr	用户使用 malloc 分配内存的地址	输入
len	内存大小	输入
dma_fd	分配内存后返回的 dma_fd	输出
cacheMode	内存存在内核态的 cache 属性。参见 SYS_CACHE_MODE_E	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

提示

- 分配的内存来自于用户 malloc 出来的区域，Fd 包含的物理内存非页对齐。
- 调用内存分配接口分配出来的内存 buffer 用一个无符号 64 位类型的 memFd 表示，memFd 在进程内唯一。跨进程需要调用 [ES_SYS_RcvMemFd](#) 进行转换得到另外进程下对应的 memFd。
- 接口内部处理用户态内存对 DMA 操作的对齐要求。
- 使用完毕后需要对 malloc 内存进行 free 操作。
- 此接口生成的 fd 通过 ES_VB_AllocIOVA 生成的 iova 以及实际物理地址都是非页对齐的，如果设备端要求页对齐，可能会影响正常使用；需要确认 SOC 的 IP 硬件能否使用这个不对齐的 FD，如果硬件有对齐约束，需要考虑首先对齐 fd 的内存。

2.3.2. ES_UserFd2MemFd

【函数体】

```
ES_S32 ES_UserFd2MemFd(ES_U64 *memFd, int fd, size_t size, int prot);
```

【描述】

将现存的 32 bit dmabuf_fd 接入 sdkmemory 管理，生成的 64bit memFd 持有与原 dmabuf_fd 相同的内存，可以使用 sdkmemory 提供的 IOVA、VA 管理接口。

【参数】

参数名称	描述	输入/输出
memFd	生成的 64bit memFd	输出
fd	原始 dmabuf_fd	输入
size	dmabuf_fd 持有的内存大小	输入
prot	dmabuf_fd 持有内存的读写权限，用于 mmap 获取 VA	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参见 错误码

【注意事项】

- 1. prot 参数与 mmap 接口的 prot 参数含义和取值一致，填 0 默认使用 PROT_READ|PROT_WRITE；当 dmabuf_fd 本身只读的时候需要指定为 PROT_READ。
- 2. 此接口生成的 memfd 需要使用 ES_SYS_MemFree 释放，但入参的 32bit fd 需要用户主动释放，只要在此接口返回成功之后 close 原生 fd 都是安全的。
- 3. 此接口产生的 memFd 跨进程调用 ES_SYS_RcvFd 时，Rcv 获取的 Fd 会丢失 prot 属性，因此不支持跨进程调用 ES_SYS_RcvFd

2.4. 数据类型

2.4.1. SYS_CACHE_MODE_E

【说明】

定义缓存块在 kernel 态或者用户态的 cache 属性。

【定义】

```
typedef enum esSYS_CACHE_MODE_E {  
    SYS_CACHE_MODE_NOCACHE = 0,  
    SYS_CACHE_MODE_CACHED = 1,  
    SYS_CACHE_MODE_LLC = 2,  
    SYS_CACHE_MODE_BUTT  
} SYS_CACHE_MODE_E;
```

【成员】

成员	描述
SYS_CACHE_MODE_NOCACHE	内核或用户态映射内存地址，CPU 访问时不经过 CPU cache
SYS_CACHE_MODE_CACHED	内核或用户态映射内存地址，CPU 访问时经过 CPU cache，需要 flush cache 保证 CPU 和设备访问内存的一致性
SYS_CACHE_MODE_LLC	内核或用户态映射内存地址，CPU 和设备访问时，都要经过 LLC，不经过 CPU 的 cache

2.4.2. VB_POOL

【说明】

定义缓存池的类型。

【定义】

```
typedef ES_S32 VB_POOL;
```

2.4.3. VB_POOL_CONFIG_S

【说明】

定义私有缓存池的属性。

【定义】

```
typedef struct esVB_POOL_CONFIG_S {
    ES_U64 blkSize;
    ES_U32 blkCnt;
    SYS_CACHE_MODE_E enRemapMode;
    ES_CHAR mmzName[ES_MAX_MMZ_NAME_LEN];
} VB_POOL_CONFIG_S;
```

【成员】

成员	描述
blkSize	缓存块大小，以 Byte 为单位
blkCnt	每个缓存池的缓存块个数
enRemapMode	VB 的内核态虚拟地址映射模式
mmzName	当前缓存池从哪个 MMZ 区域分配内存，参见 内存区域

2.4.4. VB_CONFIG_S

【说明】

公共缓存池或者模块缓存池的配置参数。

【定义】

```
#define ES_VB_MAX_MOD_POOL 16

typedef struct esVB_CONFIG_S {
    ES_U32 poolCnt;
    VB_POOL_CONFIG_S poolCfgs[ES_VB_MAX_MOD_POOL];
} VB_CONFIG_S;
```

【成员】

成员	描述
poolCnt	配置缓存池的个数
poolCfgs	缓存池的配置参数

2.4.5. VB_UID_E

【说明】

视频缓冲池的模块 ID 枚举类型。

【定义】

```
typedef enum esVB_UID_E {
    VB_UID_PRIVATE = 0,
    VB_UID_COMMON,
    VB_UID_VI,
    VB_UID_VO,
    VB_UID_VPS,
    VB_UID_VENC,
    VB_UID_VDEC,
    VB_UID_HAE,
    VB_UID_USER,
    VB_UID_BUTT,
    VB_UID_MAX,
} VB_UID_E;
```

2.4.6. EIC770X_LOGICAL_MEM_NODE_E

【说明】

逻辑 node ID 枚举类型。

【定义】

```
typedef enum {
    EIC770X_LOGICAL_FLAT_MEM_NODE_0 = 0,
    EIC770X_LOGICAL_FLAT_MEM_NODE_1 = 1,
    EIC770X_LOGICAL_SPRAM_NODE_0 = 500,
    EIC770X_LOGICAL_SPRAM_NODE_1 = 501,
    EIC770X_LOGICAL_INTERLEAVE_MEM_NODE = 1000,
} EIC770X_LOGICAL_MEM_NODE_E;
```

【成员】

成员	描述
EIC770X_LOGICAL_FLAT_MEM_NODE_0	DIE 0 对应的 node ID 逻辑编码
EIC770X_LOGICAL_FLAT_MEM_NODE_1	DIE 1 对应的 node ID 逻辑编码
EIC770X_LOGICAL_SPRAM_NODE_0	DIE 0 上 SPRAM 的 node ID 逻辑编码
EIC770X_LOGICAL_SPRAM_NODE_1	DIE 1 上 SPRAM 的 node ID 逻辑编码
EIC770X_LOGICAL_INTERLEAVE_MEM_NODE	INTERLEAVE 模式对应 node ID 逻辑编码

3. Python API 接口说明

Python 接口封装在 `es_py.memory` 模块中，提供了与 Memory 模块 C 接口对应的系统内存管理与视频缓冲管理能力，同时保留 Python 异常与类型封装的使用习惯。

导入方式：

```
from es_py import memory
```

【Python SDK 特性】

特性	Python SDK	说明
错误处理	抛出 <code>ESMemoryError</code> 异常	使用 Python 异常机制，代码更简洁
内存映射	返回 <code>(memoryview, capsule)</code> 元组	<code>memoryview</code> 支持切片操作
IOVA 分配	返回 <code>capsule</code> 对象	封装物理地址，类型安全
参数命名	<code>snake_case</code> 风格	符合 PEP 8 规范
线程安全	自动释放 GIL，支持多线程并发	无需手动处理线程同步

【最佳实践】

1. 始终使用 `try/except` 捕获 `ESMemoryError` 异常。
2. `memory_map()` 与 `memory_unmap()` 必须成对调用。
3. 使用 `memoryview` 对象访问内存，享受 Python 切片特性。
4. 跨进程传递内存时使用 `memory_retrieve_fd()` 转换文件描述符。

3.1. API 详细说明

3.1.1. `memory_alloc()`

【函数原型】

```
mem_fd = memory.memory_alloc(  
    cache_type: SYS_CACHE_MODE_E,  
    p_str_mmb: str,  
    str_zone: str,  
    len: int  
) -> int
```

【功能描述】

从指定的内存区域分配内存块。

【参数】

参数名称	类型	描述	输入/输出
cache_type	SYS_CACHE_MODE_E	内存在内核空间的 Cache 属性 • SYS_CACHE_MODE_NOCACHE: 内核空间 CPU 访问时，不经过 CPU 的 Cache • SYS_CACHE_MODE_CACHED: 内核空间 CPU 访问时，经过 CPU 的 Cache • SYS_CACHE_MODE_LLC: 内核空间 CPU 访问时，经过 LLC，不经过 CPU 的 Cache；设备通过 DMA 地址访问时，经过 LLC	输入
p_str_mmb	str	MMB 名称的字符串，用于 proc 信息显示	输入
str_zone	str	分配内存所对应的区域。str_zone 可以是 MMZ 分区（例如"mmz_nid_0_part_0"），也可以是"system"、"linux,cma"或"llc_spram0"（来自 SPRAM）。若传入 None，则默认使用"mmz_nid_0_part_0"	输入
len	int	内存大小（字节）	输入

【返回值】

返回值	描述
mem_fd (int)	成功时返回内存块的文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.2. memory_map()

【函数原型】

```
mem_view, capsule = memory.memory_map(  
    mem_fd: int,  
  
    size: int,  
    cache_type: SYS_CACHE_MODE_E  
) -> Tuple[memoryview, capsule]
```

【功能描述】

映射指定内存块，得到用户空间虚拟地址。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	需要映射的内存块文件描述符	输入
size	int	需要映射的内存大小（字节）	输入
cache_type	SYS_CACHE_MODE_E	内存在用户空间的 Cache 属性 • SYS_CACHE_MODE_NOCACHE: 用户空间 CPU 访问时，不经过 CPU 的 Cache • SYS_CACHE_MODE_CACHED: 用户空间 CPU 访问时，经过 CPU 的 Cache • SYS_CACHE_MODE_LLC: 用户空间 CPU 访问时，经过 LLC，不经过 CPU 的 Cache	输入

【返回值】

返回元组 (memoryview, capsule) 。

- memoryview : 可读写的内存视图对象，用于访问映射的内存
- capsule : 内部指针封装，传递给 memory_unmap() 使用

【异常】

失败时抛出 ESMemoryError 异常。

提示

Python 版本中，memory_map() 返回的是 memoryview 对象而非 C 指针，可以直接进行切片和读写操作。

3.1.3. memory_unmap()

【函数原型】

```
memory.memory_unmap(  
    p_vir_addr: int,  
    size: int  
) -> int
```

【功能描述】

反映射指定内存块用户空间虚拟地址。

【参数】

参数名称	类型	描述	输入/输出
p_vir_addr	int	需要反映射的用户空间虚拟地址 (可由 memory_map() 返回的 capsule 转换得到)	输入
size	int	反映射大小 (字节)	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码 (通常为 0)
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.4. memory_flush_cache()

【函数原型】

```
memory.memory_flush_cache(  
    mem_fd: int  
) -> int
```

【功能描述】

刷新 Cache 里的内容到内存并使 Cache 里的内容无效。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	待操作内存所对应的文件描述符	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.5. memory_free()

【函数原型】

```
memory.memory_free(  
    mem_fd: int  
) -> int
```

【功能描述】

释放通过 memory.memory_alloc 申请得到的内存块。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	待释放内存所对应的文件描述符	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.6. memory_split_fd()

【函数原型】

```
splited_mem_fd = memory.memory_split_fd(  
    mem_fd: int,  
    offset: int,  
    len: int  
) -> int
```

【功能描述】

从一个内存块的 offset 偏移处，按照指定的 len 大小，生成一个新的内存块。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	源内存块的文件描述符	输入
offset	int	在源内存块中的偏移地址（需按 PAGE_SIZE 对齐）	输入
len	int	分割出的内存块大小（需按 PAGE_SIZE 对齐）	输入

【返回值】

返回值	描述
splited_mem_fd (int)	成功时返回新分割的内存块文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.7. memory_retrieve_fd()**【函数原型】**

```
rcvd_mem_fd = memory.memory_retrieve_fd(  
    snd_mem_fd: int  
) -> int
```

【功能描述】

接收发送端传递过来的内存文件描述符。

【参数】

参数名称	类型	描述	输入/输出
snd_mem_fd	int	发送端内存文件描述符	输入

【返回值】

返回值	描述
rcvd_mem_fd (int)	成功时返回接收到的内存文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.8. get_memory_size()**【函数原型】**

```
size = memory.get_memory_size(  
    mem_fd: int  
) -> int
```

【功能描述】

获取指定内存块的大小。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存块的文件描述符	输入

【返回值】

返回值	描述
size (int)	成功时返回内存块大小（字节）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.9. retrieve_node_by_fd()

【函数原型】

```
nid = memory.retrieve_node_by_fd(  
    mem_fd: int  
) -> EIC770X_LOGICAL_MEM_NODE_E
```

【功能描述】

获取指定内存块所在的 Node ID。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存块的文件描述符	输入

【返回值】

返回值	描述
nid (EIC770X_LOGICAL_MEM_NODE_E)	成功时返回逻辑内存节点 ID
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.10. retrieve_node_by_vaddr()

【函数原型】

```
nid = memory.retrieve_node_by_vaddr(  
    p_vir_addr: int  
) -> EIC770X_LOGICAL_MEM_NODE_E
```

【功能描述】

通过缓冲块虚拟地址获取缓存块所在的 Node ID。

【参数】

参数名称	类型	描述	输入/输出
p_vir_addr	int	用户空间虚拟地址	输入

【返回值】

返回值	描述
nid (EIC770X_LOGICAL_MEM_NODE_E)	成功时返回逻辑内存节点 ID
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.11. memory_flush_cache_all_by_nid()

【函数原型】

```
memory.memory_flush_cache_all_by_nid(  
    nid: EIC770X_LOGICAL_MEM_NODE_E  
) -> int
```

【功能描述】

将指定 Node ID 所有 L3 缓存内容刷入主存并将缓存内容无效。

【参数】

参数名称	类型	描述	输入/输出
nid	EIC770X_LOGICAL_MEM_NODE_E	逻辑内存节点 ID	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.12. memory_flush_cache_all()

【函数原型】

```
memory.memory_flush_cache_all() -> int
```

【功能描述】

将所有 L3 缓存内容刷入主存并将缓存内容无效。

【参数】

无

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.13. `get_internal_fd()`

【函数原型】

```
fd = memory.get_internal_fd(  
    mem_fd: int  
) -> int
```

【功能描述】

获取内存文件描述符对应的内部文件描述符。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存块的文件描述符	输入

【返回值】

返回值	描述
fd (int)	成功时返回内部文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.14. `get_internal_memory_fd()`

【函数原型】

```
mem_fd = memory.get_internal_memory_fd(  
    fd: int  
) -> int
```

【功能描述】

获取内部文件描述符对应的内存文件描述符。

【参数】

参数名称	类型	描述	输入/输出
fd	int	内部文件描述符	输入

【返回值】

返回值	描述
mem_fd (int)	成功时返回内存文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.15. `video_buffer_create_pool()`

【函数原型】

```
pool_id = memory.video_buffer_create_pool(  
    pst_vb_pool_cfg: VB_POOL_CONFIG_S  
) -> int
```

【功能描述】

从指定的内存区域创建一个私有视频缓存池。

【参数】

参数名称	类型	描述	输入/输出
pst_vb_pool_cfg	VB_POOL_CONFIG_S	视频缓冲池配置结构体	输入

【返回值】

返回值	描述
pool_id (int)	成功时返回创建的缓冲池 ID
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.16. video_buffer_destroy_pool()

【函数原型】

```
memory.video_buffer_destroy_pool(  
    pool_id: int  
) -> int
```

【功能描述】

销毁一个私有视频缓存池。

【参数】

参数名称	类型	描述	输入/输出
pool_id	int	待销毁的缓冲池 ID	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.17. video_buffer_set_config()

【函数原型】

```
memory.video_buffer_set_config(  
    p_vb_config: VB_CONFIG_S  
) -> int
```

【功能描述】

设置公共视频缓存池属性。

【参数】

参数名称	类型	描述	输入/输出
p_vb_config	VB_CONFIG_S	视频缓冲配置结构体	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.18. video_buffer_get_config()

【函数原型】

```
vb_config = memory.video_buffer_get_config() -> VB_CONFIG_S
```

【功能描述】

获取公共视频缓存池属性。

【参数】

无

【返回值】

返回值	描述
vb_config (VB_CONFIG_S)	成功时返回视频缓冲配置结构体
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.19. video_buffer_init()

【函数原型】

```
memory.video_buffer_init() -> int
```

【功能描述】

创建公共视频缓存池。

【参数】

无

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）

返回值	描述
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.20. video_buffer_exit()

【函数原型】

```
memory.video_buffer_exit() -> int
```

【功能描述】

销毁公共视频缓存池。

【参数】

无

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.21. video_buffer_set_mod_pool_config()

【函数原型】

```
memory.video_buffer_set_mod_pool_config(  
    en_vb_uid: VB_UID_E,  
    p_vb_config: VB_CONFIG_S  
) -> int
```

【功能描述】

设置模块视频缓存池属性。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入
p_vb_config	VB_CONFIG_S	模块缓冲池配置结构体	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.22. video_buffer_get_mod_pool_config()

【函数原型】

```
vb_config = memory.video_buffer_get_mod_pool_config(  
    en_vb_uid: VB_UID_E  
) -> VB_CONFIG_S
```

【功能描述】

获取模块视频缓存池属性。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入

【返回值】

返回值	描述
vb_config (VB_CONFIG_S)	成功时返回模块缓冲池配置结构体
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.23. video_buffer_mod_pool_init()

【函数原型】

```
memory.video_buffer_mod_pool_init(  
    en_vb_uid: VB_UID_E  
) -> int
```

【功能描述】

创建模块视频缓存池。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.24. video_buffer_mod_pool_exit()

【函数原型】

```
memory.video_buffer_mod_pool_exit(  
    en_vb_uid: VB_UID_E  
) -> int
```

【功能描述】

销毁模块视频缓存池。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.25. video_buffer_get_mod_block()

【函数原型】

```
mem_fd = memory.video_buffer_get_mod_block(  
    en_vb_uid: VB_UID_E,  
    blk_size: int,  
    str_zone: str = ""  
) -> int
```

【功能描述】

从模块缓存池中获取一个缓存块。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入
blk_size	int	申请的内存块大小（字节）	输入
str_zone	str	MMZ 分区名称，默认为空字符串	输入

【返回值】

返回值	描述
mem_fd (int)	成功时返回内存文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.26. video_buffer_get_block()

【函数原型】

```
mem_fd = memory.video_buffer_get_block(  
    pool_id: int,  
    blk_size: int,  
    str_zone: str = ""  
) -> int
```

【功能描述】

获取一个缓存块。

【参数】

参数名称	类型	描述	输入/输出
pool_id	int	缓冲池 ID	输入
blk_size	int	申请的内存块大小（字节）	输入
str_zone	str	MMZ 分区名称，默认为空字符串	输入

【返回值】

返回值	描述
mem_fd (int)	成功时返回内存文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.27. video_buffer_release_block()

【函数原型】

```
memory.video_buffer_release_block(  
    mem_fd: int  
) -> int
```

【功能描述】

释放一个已经获取的缓存块。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	待释放的内存文件描述符	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.28. video_buffer_alloc_iova()

【函数原型】

```
iova_capsule = memory.video_buffer_alloc_iova(
    mem_fd: int,
    en_vb_uid: VB_UID_E
) -> capsule
```

【功能描述】

获取一个缓存块的 IOVA 地址。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存文件描述符	输入
en_vb_uid	VB_UID_E	模块唯一标识	输入

【返回值】

返回值	描述
iova_capsule (capsule)	成功时返回 IOVA 地址的 capsule 对象
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.29. video_buffer_iova2_handle()

【函数原型】

```
mem_fd = memory.video_buffer_iova2_handle(
    en_vb_uid: VB_UID_E,
    p_iova: capsule
) -> int
```

【功能描述】

获取模块 IOVA 地址对应的内存文件描述符。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入
p_iova	capsule	IOVA 地址的 capsule 对象	输入

【返回值】

返回值	描述
mem_fd (int)	成功时返回内存文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.30. iova_capsule_to_int()

【函数原型】

```
iova = memory.iova_capsule_to_int(  
    iova_capsule: capsule  
) -> int
```

【功能描述】

将 video_buffer_alloc_iova() 返回的 capsule 转换为整数形式的 IOVA 地址。

【参数】

参数名称	类型	描述	输入/输出
iova_capsule	capsule	IOVA 地址的 capsule 对象	输入

【返回值】

返回值	描述
iova (int)	成功时返回 IOVA 地址的整数值
异常	失败时抛出 ESMemoryError 或参数异常

3.1.31. video_buffer_free_iova()

【函数原型】

```
memory.video_buffer_free_iova(  
    en_vb_uid: VB_UID_E,  
    mem_fd: int  
) -> int
```

【功能描述】

释放缓存块的 IOVA 地址空间。

【参数】

参数名称	类型	描述	输入/输出
en_vb_uid	VB_UID_E	模块唯一标识	输入
mem_fd	int	内存文件描述符	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.32. video_buffer_handle2_pool_id()

【函数原型】

```
pool_id = memory.video_buffer_handle2_pool_id(
    mem_fd: int
) -> int
```

【功能描述】

获取一个缓存块所在缓存池的 ID。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存文件描述符	输入

【返回值】

返回值	描述
pool_id (int)	成功时返回缓冲池 ID
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.33. video_buffer_memory_map_pool()

【函数原型】

```
memory.video_buffer_memory_map_pool(
    pool_id: int
) -> int
```

【功能描述】

为一个视频缓存池映射用户空间虚拟地址。

【参数】

参数名称	类型	描述	输入/输出
pool_id	int	缓冲池 ID	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.34. video_buffer_memory_unmap_pool()

【函数原型】

```
memory.video_buffer_memory_unmap_pool(  
    pool_id: int  
) -> int
```

【功能描述】

解除一个视频缓存池的用户空间映射。

【参数】

参数名称	类型	描述	输入/输出
pool_id	int	缓冲池 ID	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.35. video_buffer_get_block_vir_addr()

【函数原型】

```
virt_addr = memory.video_buffer_get_block_vir_addr(  
    pool_id: int,  
    mem_fd: int  
) -> int
```

【功能描述】

获取一个视频缓存池中的缓存块的用户空间虚拟地址。

【参数】

参数名称	类型	描述	输入/输出
pool_id	int	缓冲池 ID	输入
mem_fd	int	内存文件描述符	输入

【返回值】

返回值	描述
virt_addr (int)	成功时返回虚拟地址
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.36. video_buffer_register_block_ref_cnt_notify_callback()

【函数原型】

```
memory.video_buffer_register_block_ref_cnt_notify_callback(  
    mem_fd: int,  
    notify_refcnt: int,  
    callback: callable,  
    callback_arg: int  
) -> int
```

【功能描述】

为指定 `mem_fd` 注册回调。当引用计数减小到 `notify_refcnt` 时触发回调。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	需要注册通知的内存文件描述符	输入
notify_refcnt	int	触发回调的目标引用计数	输入
callback	callable	引用计数通知回调函数	输入
callback_arg	int	回调函数的用户参数（指针/句柄整数形式）	输入

【返回值】

返回值	描述
ret (int)	成功时返回状态码（通常为 0）
异常	失败时抛出 ESMemoryError 异常，参见 错误码

提示

`video_buffer_register_block_ref_cnt_notify_callback()` 仅在构建版本满足 `ESSDK_VERSION ≥ 20250630` 时可用。

3.1.37. video_buffer_get_block_ref_count()

【函数原型】

```
blk_file_cnt = memory.video_buffer_get_block_ref_count(  
    mem_fd: int  
) -> int
```

【功能描述】

获取指定内存块的引用计数。

【参数】

参数名称	类型	描述	输入/输出
mem_fd	int	内存文件描述符	输入

【返回值】

返回值	描述
blk_file_cnt (int)	成功时返回引用计数
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.1.38. malloc_to_memory_fd()

【函数原型】

```
dma_fd = memory.malloc_to_memory_fd(  
    orig_ptr: int,  
    length: int,  
    cache_type: SYS_CACHE_MODE_E  
) -> int
```

【功能描述】

将用户空间 malloc 的内存转换为内存块。

【参数】

参数名称	类型	描述	输入/输出
orig_ptr	int	原始指针地址	输入
length	int	内存长度（字节）	输入
cache_type	SYS_CACHE_MODE_E	Cache 类型	输入

【返回值】

返回值	描述
dma_fd (int)	成功时返回 DMA 文件描述符
异常	失败时抛出 ESMemoryError 异常，参见 错误码

3.2. 数据类型

定义 Memory 模块使用的 Python 数据类型和结构体。所有类型定义遵循 Python 类型注解规范，使用 `@dataclass` 装饰器和 `Enum` 枚举类型。

类型名称	分类	说明
SYS_CACHE_MODE_E	枚举类	定义缓存块在内核态或用户空间的 Cache 属性
VB_POOL	类型别名	缓存池 ID 类型 (int)
VB_POOL_CONFIG_S	数据类	定义私有缓存池的属性
VB_CONFIG_S	数据类	定义公共缓存池或模块缓存池的配置参数
VB_UID_E	枚举类	定义视频缓冲池的模块 ID 枚举类型
EIC770X_LOGICAL_MEM_NODE_E	枚举类	定义逻辑 Node ID 枚举类型

【类型导入】

```
from es_py.sys_types import SYS_CACHE_MODE_E
from es_py.video_buffer_types import (
    VB_POOL,
    VB_POOL_CONFIG_S,
    VB_CONFIG_S,
    VB_UID_E,
    EIC770X_LOGICAL_MEM_NODE_E
)
```

3.2.1. SYS_CACHE_MODE_E

【说明】

定义缓存块在内核态或用户空间的 Cache 属性

【定义】

```
class SYS_CACHE_MODE_E(Enum):
    SYS_CACHE_MODE_NOCACHE = 0 # 不经过 CPU Cache
    SYS_CACHE_MODE_CACHED = 1 # 经过 CPU Cache
    SYS_CACHE_MODE_LLC = 2 # 经过 LLC, 不经过 CPU Cache
```

【成员】

成员	描述
SYS_CACHE_MODE_NOCACHE	内核或用户空间映射内存地址，CPU 访问时不经过 CPU Cache
SYS_CACHE_MODE_CACHED	内核或用户空间映射内存地址，CPU 访问时经过 CPU Cache，需要 flush cache 保证 CPU 和设备访问内存的一致性
SYS_CACHE_MODE_LLC	内核或用户空间映射内存地址，CPU 和设备访问时，都要经过 LLC，不经过 CPU 的 Cache

3.2.2. VB_POOL

【说明】

定义缓存池的类型

【定义】

```
VB_POOL = int # 缓存池 ID 类型
```

3.2.3. VB_POOL_CONFIG_S

【说明】

定义私有缓存池的属性

【定义】

```
class VB_POOL_CONFIG_S:
    blkSize: int # 缓存块大小 (字节)
    blkCnt: int # 每个缓存池的缓存块个数
    enRemapMode: SYS_CACHE_MODE_E # VB 内核空间虚拟地址映射模式
```

mmzName: str # 当前缓存池从哪个 MMZ 区域分配内存

【成员】

成员	类型	描述
blkSize	int	缓存块大小（字节）
blkCnt	int	每个缓存池的缓存块个数
enRemapMode	SYS_CACHE_MODE_E	VB 的内核空间虚拟地址映射模式
mmzName	str	当前缓存池从哪个 MMZ 区域分配内存

3.2.4. VB_CONFIG_S

【说明】

定义公共缓存池或模块缓存池的配置参数

【定义】

```
class VB_CONFIG_S:
    poolCnt: int      # 配置缓存池的个数
    poolCfgs: CArrayProxy[VB_POOL_CONFIG_S] # 缓存池的配置参数数组（通过索引访问）
```

【成员】

成员	类型	描述
poolCnt	int	配置缓存池的个数
poolCfgs	CArrayProxy VB_POOL_CONFIG_S	缓存池的配置参数数组，支持索引访问

3.2.5. VB_UID_E

【说明】

定义视频缓冲池的模块 ID 枚举类型

【定义】

```
class VB_UID_E(Enum):
    VB_UID_PRIVATE = 0 # 私有缓冲池
    VB_UID_COMMON = 1 # 公共缓冲池
    VB_UID_VI = 2      # VI 模块
    VB_UID_VO = 3      # VO 模块
    VB_UID_VPS = 4      # VPS 模块
    VB_UID_VENC = 5      # VENC 模块
    VB_UID_VDEC = 6      # VDEC 模块
    VB_UID_HAE = 7      # HAE 模块
    VB_UID_USER = 8      # 用户自定义
    VB_UID_BUTT = 9      # 边界值
    VB_UID_MAX = 10      # 最大值
```

3.2.6. EIC770X_LOGICAL_MEM_NODE_E

【说明】

定义逻辑 Node ID 枚举类型

【定义】

```
class EIC770X_LOGICAL_MEM_NODE_E(Enum):
    EIC770X_LOGICAL_FLAT_MEM_NODE_0 = 0      # DIE 0 对应的 Node ID 逻辑编码
    EIC770X_LOGICAL_FLAT_MEM_NODE_1 = 1      # DIE 1 对应的 Node ID 逻辑编码
    EIC770X_LOGICAL_SPRAM_NODE_0 = 500       # DIE 0 上 SPRAM 的 Node ID 逻辑编码
    EIC770X_LOGICAL_SPRAM_NODE_1 = 501       # DIE 1 上 SPRAM 的 Node ID 逻辑编码
    EIC770X_LOGICAL_INTERLEAVE_MEM_NODE = 1000 # INTERLEAVE 模式对应 Node ID 逻辑编码
```

【成员】

成员	描述
EIC770X_LOGICAL_FLAT_MEM_NODE_0	DIE 0 对应的 Node ID 逻辑编码
EIC770X_LOGICAL_FLAT_MEM_NODE_1	DIE 1 对应的 Node ID 逻辑编码
EIC770X_LOGICAL_SPRAM_NODE_0	DIE 0 上 SPRAM 的 Node ID 逻辑编码
EIC770X_LOGICAL_SPRAM_NODE_1	DIE 1 上 SPRAM 的 Node ID 逻辑编码
EIC770X_LOGICAL_INTERLEAVE_MEM_NODE	INTERLEAVE 模式对应 Node ID 逻辑编码

4. 错误码

Memory 相关 API 错误码如下表所示。

4.1. SYS 模块错误码

错误码	宏定义	描述
0xA0026003	ES_SYS_ERROR_ILLEGAL_PARAM	参数非法
0xA0026006	ES_SYS_ERROR_NULL_PTR	使用空指针
0xA0026007	ES_SYS_ERROR_NOT_CONFIG	配置属性前尝试初始化系统
0xA0026008	ES_SYS_ERROR_NOT_SUPPORT	操作或类型不支持
0xA0026009	ES_SYS_ERROR_NOT_PERM	操作不允许
0xA002600C	ES_SYS_ERROR_NOMEM	内存分配失败
0xA002600D	ES_SYS_ERROR_NOBUF	缓冲区分配失败
0xA0026011	ES_SYS_ERROR_BADADDR	地址错误
0xA0026013	ES_SYS_ERROR_SIZE_NOT_ENOUGH	缓冲区大小不足
0xA0026040	ES_SYS_ERROR_DEV_NOT_FOUND	设备未找到
0xA0026041	ES_SYS_ERROR_DEV_NOT_OPEN	设备未打开
0xA0026042	ES_SYS_ERROR_OPERATION_FAIL	操作失败
0xA0026043	ES_SYS_ERROR_FLUSH_CACHE_FAIL	刷新缓存失败
0xA0026044	ES_SYS_ERROR_MEM_FREE_FAIL	释放内存失败
0xA0026045	ES_SYS_ERROR_GET_SIZE_FAIL	获取大小失败

4.2. VB 模块错误码

错误码	宏定义	描述
0xA0016003	ES_VB_ERROR_ILLEGAL_PARAM	参数非法
0xA0016006	ES_VB_ERROR_NULL_PTR	使用空指针
0xA0016007	ES_VB_ERROR_NOT_CONFIG	配置属性前尝试初始化系统
0xA0016008	ES_VB_ERROR_NOT_SUPPORT	操作或类型不支持
0xA0016009	ES_VB_ERROR_NOT_PERM	操作不允许
0xA001600C	ES_VB_ERROR_NOMEM	内存分配失败
0xA001600D	ES_VB_ERROR_NOBUF	缓冲区分配失败
0xA0016011	ES_VB_ERROR_BADADDR	地址错误
0xA0016013	ES_VB_ERROR_SIZE_NOT_ENOUGH	缓冲区大小不足
0xA0016040	ES_VB_ERROR_DEV_NOT_FOUND	设备未找到
0xA0016041	ES_VB_ERROR_DEV_NOT_OPEN	设备未打开
0xA0016042	ES_VB_ERROR_OPERATION_FAIL	操作失败

错误码	宏定义	描述
0xA0016043	ES_VB_ERROR_FLUSH_CACHE_FAIL	刷新缓存失败
0xA0016044	ES_VB_ERROR_CREAT_POOL_FAIL	创建内存池失败
0xA0016045	ES_VB_ERROR_DESTROY_POOL_FAIL	销毁内存池失败
0xA0016046	ES_VB_ERROR_SET_CFG_FAIL	设置模块池配置失败
0xA0016047	ES_VB_ERROR_GET_CFG_FAIL	获取模块池配置失败
0xA0016048	ES_VB_ERROR_INIT_CFG_FAIL	初始化模块池失败
0xA0016049	ES_VB_ERROR_UNINIT_CFG_FAIL	反初始化模块池失败
0xA001604A	ES_VB_ERROR_POOL_GET_SIZE_FAIL	获取池大小失败
0xA001604B	ES_VB_ERROR_POOL_MMAP_FAIL	映射池失败
0xA001604C	ES_VB_ERROR_POOL_ILLEGAL_POOLID	池 ID 非法
0xA001604D	ES_VB_ERROR_POOL_UNMAP_FAIL	取消映射池失败
0xA001604E	ES_VB_ERROR_POOL_NOT_MAP	池未映射时获取块虚拟地址失败
0xA001604F	ES_VB_ERROR_GET_IOVA_FAIL	获取特定硬件模块 IOVA 失败
0xA0016050	ES_VB_ERROR_PUT_IOVA_FAIL	释放特定硬件模块 IOVA 失败
0xA0016051	ES_VB_ERROR_SPLIT_DMABUF_FAIL	分割 memFd 失败
0xA0016052	ES_VB_ERROR_GET_BLK_FAIL	从驱动获取块失败
0xA0016053	ES_VB_ERROR_GET_BLK_OFFSET_FAIL	获取块在池中的偏移失败
0xA0016054	ES_VB_ERROR_FLUSH_POOL_FAIL	刷新池缓存失败
0xA0016055	ES_VB_ERROR_GET_BLK_REFCNT_FAIL	获取块引用计数失败

5. 附录

5.1. 使用说明

1. 使用前确保板端运行环境下交付件中所有驱动 ko 文件已正常加载
2. 在板端运行环境下/usr/local/lib 下存在动态库 libes_memory.so
3. 各 API 的使用请参考示例代码 memory_samples.c

5.2. Proc 调试信息说明

调试信息采用了 Linux 下的 proc 文件系统，可实时反映当前系统的运行状态，所记录的信息可供问题定位及分析时使用。

1. proc/eswin/vb

在板端环境命令行下执行 cat /proc/eswin/vb，可以显示所有 MMZ 区域当前的分配和使用情况。包含所有 mmz partition 的预留内存总量，当前空闲内存总量，系统运行至今最大内存使用峰值统计，以及当前已分配的缓存池使用情况。例如在 mmz_nid_0_part_0 区域创建 2 个 pool，其中一个 pool 有 3 个 block，另一个 pool 只有 1 个 block，全部正在使用的示例如下图：

```
root@rockos-eswin:~# cat /proc/eswin/vb
Module: [VB], Build Time[xx]
-----MMZ REGION CONFIG-----
      memblock: mmz_nid_0_part_0, total size(0x27fffe000), free mem size(0x1a9ec6000), peak used size(0xe3eef000)
-----POOL CONFIG-----
      Uid 6 , PoolId 0 , blkSize 0x06000000, blkCnt 3 , RemapMode 0, mmzName mmz_nid_0_part_0, allocated blkCnt 3
      Uid 0 , PoolId 1 , blkSize 0x02000000, blkCnt 1 , RemapMode 0, mmzName mmz_nid_0_part_0, allocated blkCnt 1
root@rockos-eswin:~#
```

图 6: vb 缓存池 debug 信息示例

2. proc/llc_spram0_info

在板端环境命令行下执行 cat /proc/llc_spram0_info，可以显示 SPRAM 的总大小，及当前使用情况。

```
# cat /proc/llc_spram0_info
```

```
[root@debian:~]$ cat /proc/llc_spram0_info
SRAM pool: 4096 KiB, Available: 4096 KiB
LLC Users(0):
[root@debian:~]$
```

图 7: spram debug 信息示例

3. 查看所有分配的 dmabuf 信息

在板端环境命令行下顺序执行以下两个命令，可以查看所有分配的 dmabuf 信息。

```
# mount -t debugfs none /sys/kernel/debug
```

```
# cat /sys/kernel/debug/dma_buf/bufinfo
```

例如当前有四个 dambuf 正在被使用的情况如下图：

```
[root@debian:~]$ mount -t debugfs none /sys/kernel/debug
[root@debian:~]$ cat /sys/kernel/debug/dma_buf/bufinfo

Dma-buf Objects:
size      flags      mode      count      exp_name      ino      name
01572864   00000002   00080007   00000002   mmz_nid_0_part_0  00000084  <none>
  Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

03145728   00000002   00080007   00000004   mmz_nid_0_part_0  00000083  dst
  Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

01572864   00000002   00080007   00000002   llc_spram0        00000082  <none>
  Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

03145728   00000002   00080007   00000004   llc_spram0        00000081  src
  Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

Total 4 objects, 9437184 bytes
[root@debian:~]$
```

图 8: dmabuf 信息示例